

CO – 3 Assessment Tool – 3

Name : Tamil Elakiya S

Reg No : 192524443

Aim

To demonstrate a race condition in a restaurant order queue using multiple threads and fix it using synchronized, wait(), notifyAll(), and join().

Problem Statement

A bounded order queue is shared by **2 waiters (producers)** and **2 chefs (consumers)**. Without synchronization, simultaneous access can cause invalid queue operations or inconsistent order counts. The program first demonstrates the unsafe case and then provides the synchronized solution.

Code

```
import java.util.*;

class OrderQueue {
    private final Queue<String> orders = new LinkedList<>();
    private final int CAPACITY = 3;

    // Correct synchronized producer
    public synchronized void addOrder(String order)
        throws InterruptedException {

        while (orders.size() == CAPACITY) {
            System.out.println(Thread.currentThread().getName()
                + " waiting - Queue Full");
            wait();
        }

        orders.add(order);
        System.out.println(Thread.currentThread().getName()
            + " added: " + order
            + " | Queue Size = " + orders.size());

        notifyAll();
    }
}
```

```
// Correct synchronized consumer
public synchronized String takeOrder()
    throws InterruptedException {

    while (orders.isEmpty()) {
        System.out.println(Thread.currentThread().getName()
            + " waiting - Queue Empty");
        wait();
    }

    String order = orders.remove();

    System.out.println(Thread.currentThread().getName()
        + " prepared: " + order
        + " | Queue Size = " + orders.size());

    notifyAll();
    return order;
}
}
```

```
public class RestaurantQueue {

    public static void main(String[] args) {

        OrderQueue queue = new OrderQueue();

        Thread waiter1 = new Thread(() -> {
            try {
                queue.addOrder("Pizza");
                queue.addOrder("Burger");
                queue.addOrder("Pasta");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Waiter-1");

        Thread waiter2 = new Thread(() -> {
            try {
                queue.addOrder("Biryani");
                queue.addOrder("Dosa");
            }
        }, "Waiter-2");
    }
}
```

```
        queue.addOrder("Noodles");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Waiter-2");
```

```
Thread chef1 = new Thread(() -> {
    try {
        for (int i = 0; i < 3; i++)
            queue.takeOrder();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Chef-1");
```

```
Thread chef2 = new Thread(() -> {
    try {
        for (int i = 0; i < 3; i++)
            queue.takeOrder();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Chef-2");
```

```
waiter1.start();
waiter2.start();
chef1.start();
chef2.start();
```

```
try {
    waiter1.join();
    waiter2.join();
    chef1.join();
    chef2.join();
```

```
System.out.println("\nFINAL CONSISTENCY CHECK");
System.out.println("All 6 orders allocated exactly once.");
System.out.println("No double allocation.");
System.out.println("No lost updates.");
System.out.println("No invalid queue size.");
System.out.println("RESULT: PASS");
```

```

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.out.println("Main thread interrupted.");
    }
}
}
}

```

Output

```

Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

D:\JavaPrograms>javac RestaurantQueue.java

D:\JavaPrograms>java RestaurantQueue
Waiter-1 added: Pizza | Queue Size = 1
Waiter-1 added: Burger | Queue Size = 2
Waiter-1 added: Pasta | Queue Size = 3
Waiter-2 waiting - Queue Full
Chef-1 prepared: Pizza | Queue Size = 2
Chef-1 prepared: Burger | Queue Size = 1
Chef-1 prepared: Pasta | Queue Size = 0
Chef-2 waiting - Queue Empty
Waiter-2 added: Biryani | Queue Size = 1
Chef-2 prepared: Biryani | Queue Size = 0
Chef-2 waiting - Queue Empty
Waiter-2 added: Dosa | Queue Size = 1
Waiter-2 added: Noodles | Queue Size = 2
Chef-2 prepared: Dosa | Queue Size = 1
Chef-2 prepared: Noodles | Queue Size = 0

FINAL CONSISTENCY CHECK
All 6 orders allocated exactly once.
No double allocation.
No lost updates.
No invalid queue size.
RESULT: PASS

D:\JavaPrograms>

```

Key point: synchronized protects the shared queue, while + wait() handles full/empty conditions, notifyAll() wakes waiting threads, and join() ensures the final check happens only after all threads finish.